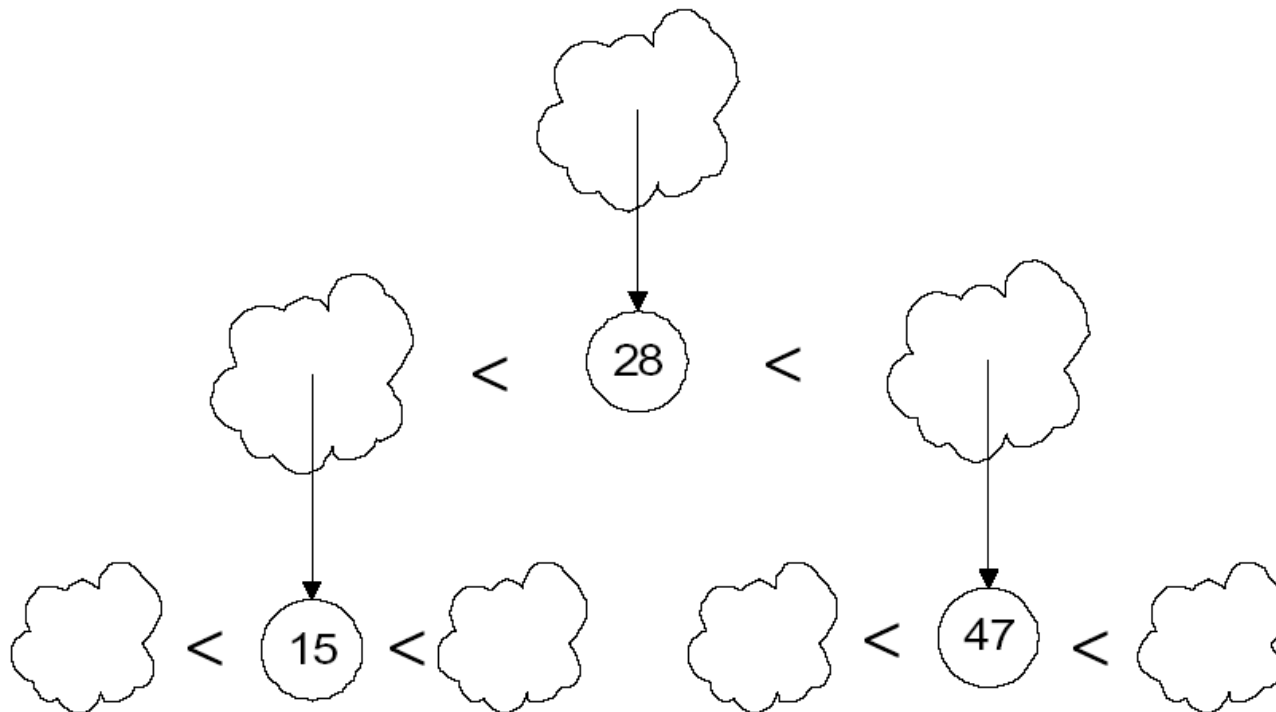


Hızlı Sıralama (Quick-Sort)

- Bu algoritma Böl-Yönet yaklaşımına dayanmaktadır.

Hızlı Sıralama



Bir pivot eleman seçilir. Pivottan küçük olanlar ve büyük olanlar olarak dizi ikiye bölünür.
Herbir taraf özyineli olarak sıralanır.

Hızlı sıralama (Quicksort) Algoritması

Tamsayılardan oluşan n elemanlı bir dizi verilmiş olsun.

- If {dizi sadece bir elemandan oluşuyorsa} , return
- Else
 - *pivot* olarak kullanmak için bir eleman seç
 - Diziyi aşağıdaki gibi iki alt dizi halinde böl:
 - *Pivot*'a eşit yada pivottan küçük olanlar
 - *Pivottan* büyük olanlar
 - İki alt diziyi Hızlı sırala
 - Return sonuç dizi

Örnek

Verilen diziyi algoritmaya göre sıralayalım

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

Pivot Eleman seç

- Pivot eleman değişik şekillerde seçilebilir. Bu örnekte ilk eleman pivot olarak seçilecektir.

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

Diziyi bölme

- Pivottan küçük yada eşit olanlardan oluşan bir altdizi($\leq$ pivot)
- Pivottan büyük olan elemanlardan oluşan bir dizi($>$ pivot)
- **Altdiziler orijinal dizide tutulur.**
- **Bölme işlemini diziler parçalanamayacak hale gelene kadar rekürsif olarak tekrarla**

pivot_index = 0

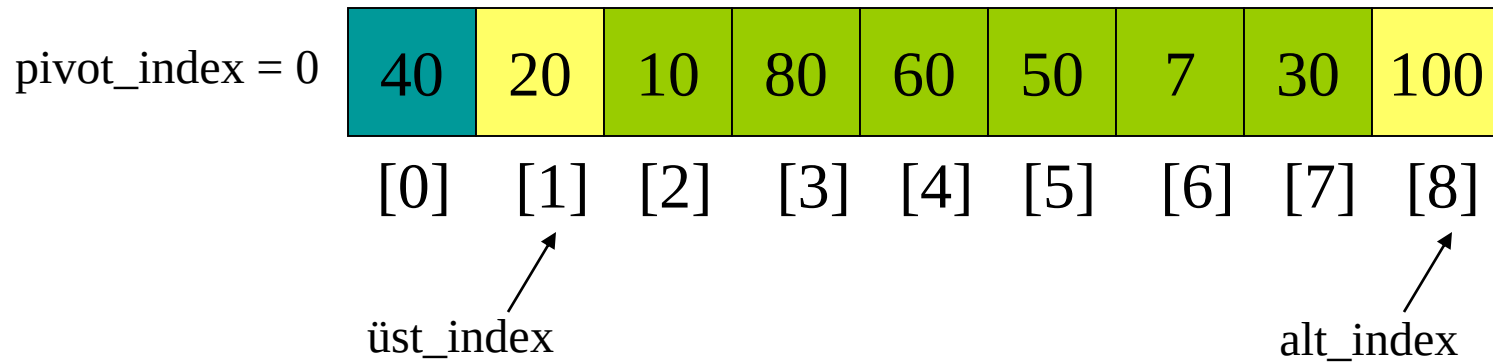
40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

[0] [1] [2] [3] [4] [5] [6] [7] [8]

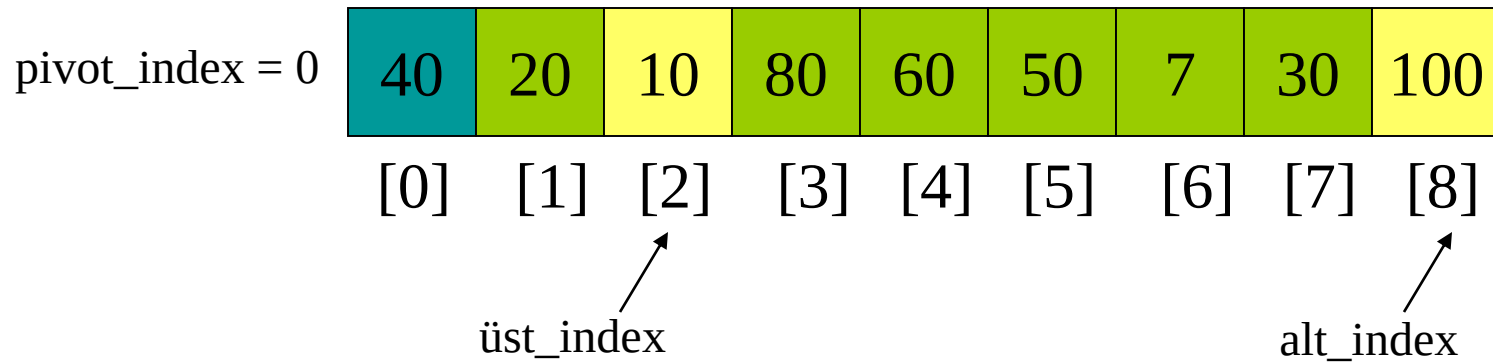
üst_index

alt_index

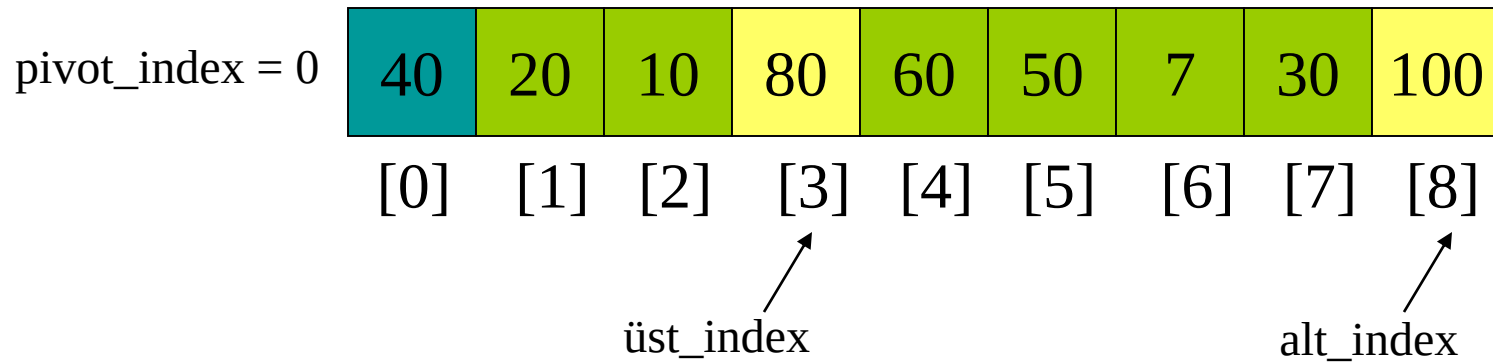
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 $++\text{üst_index}$



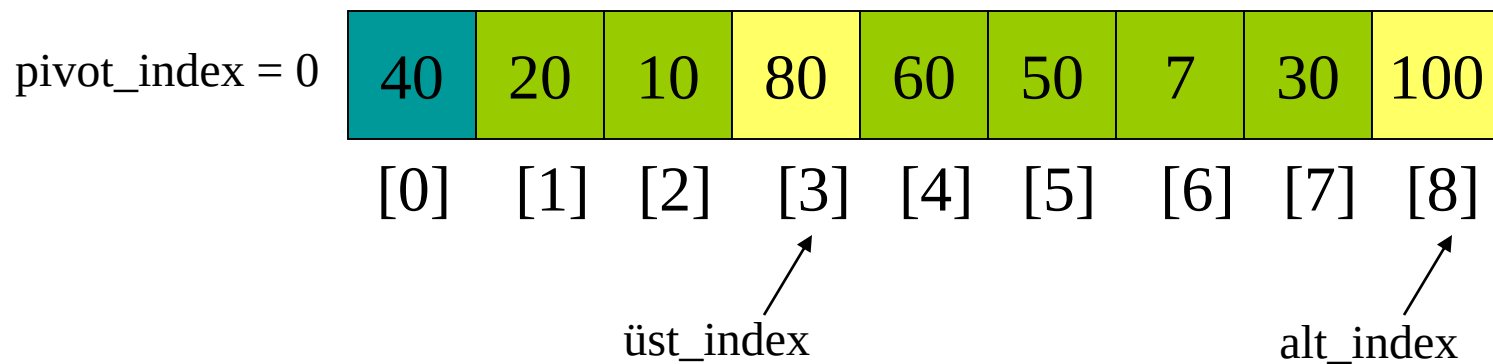
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index



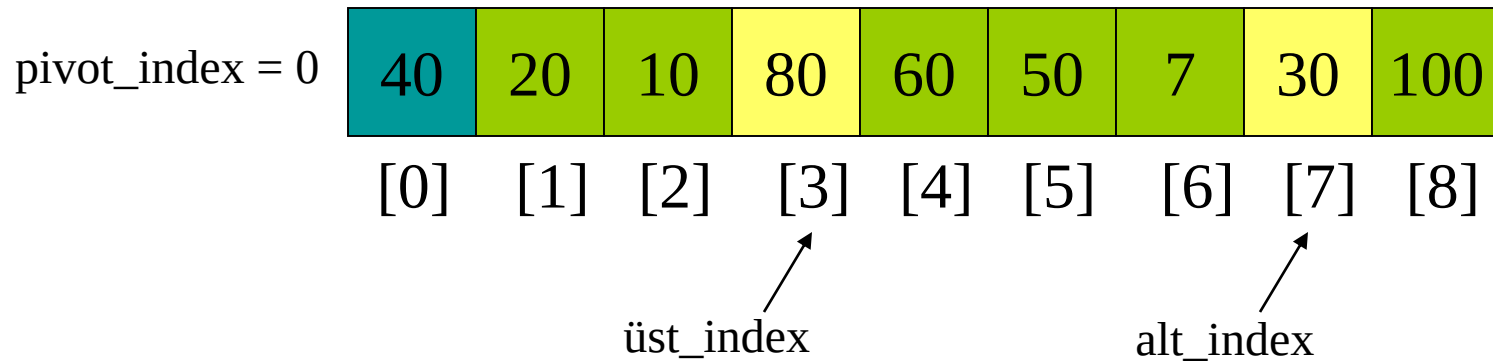
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index



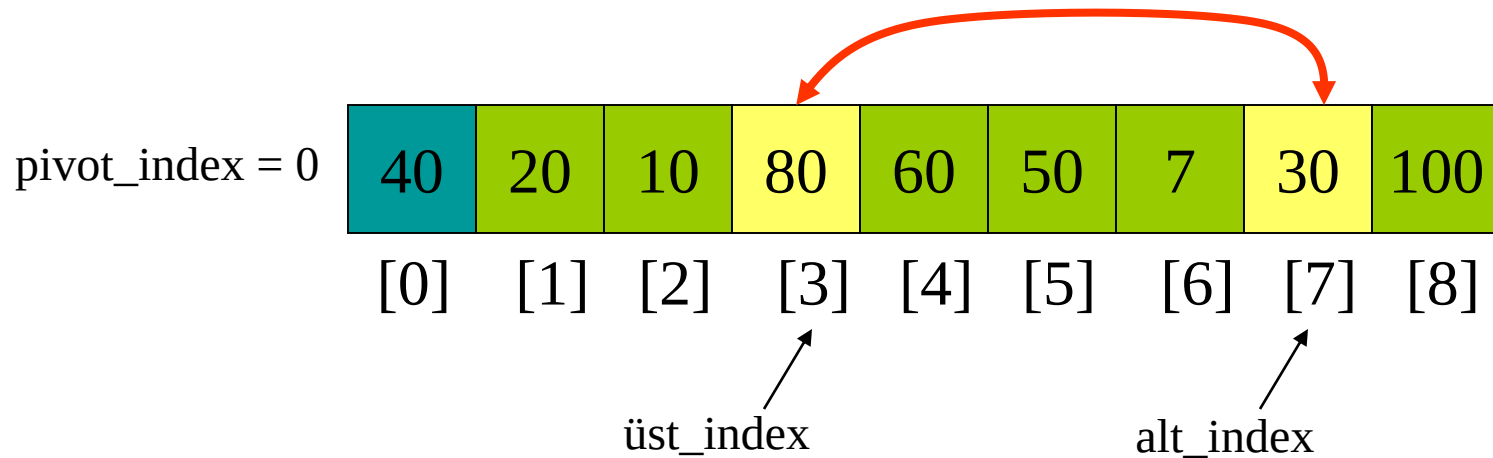
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index



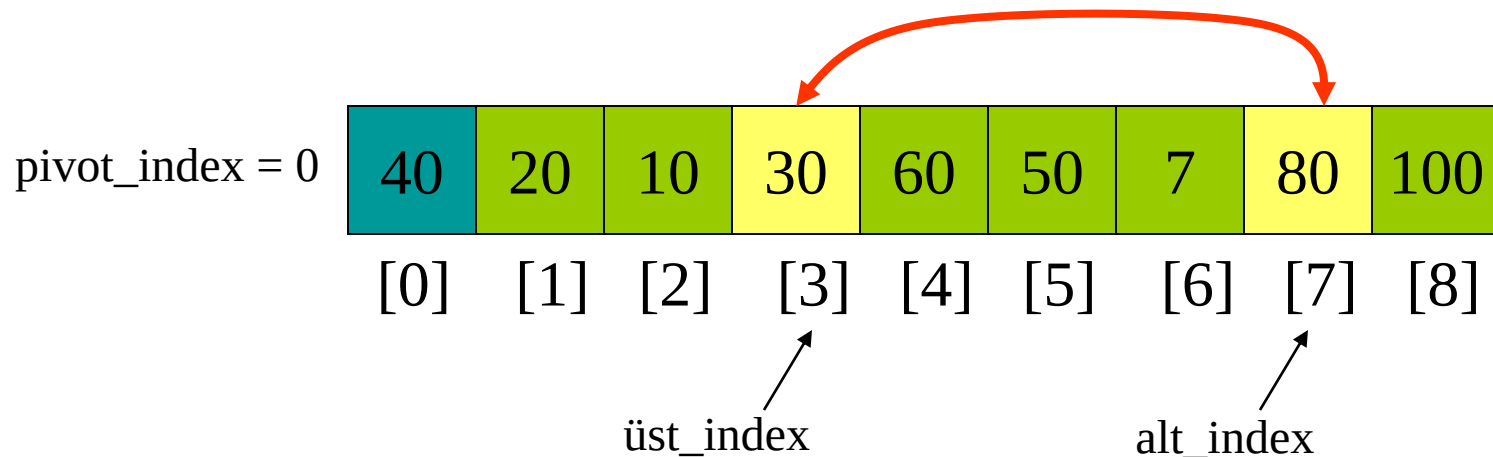
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index



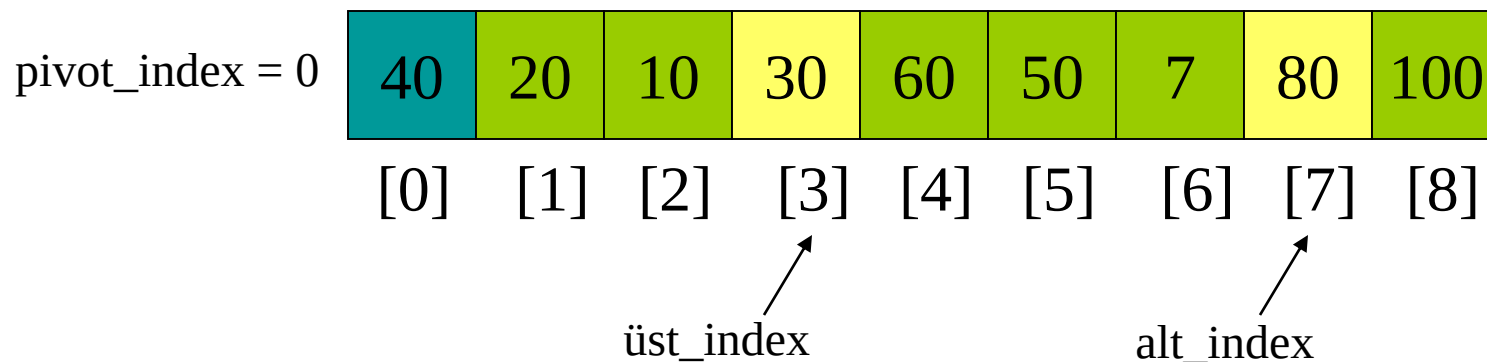
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$



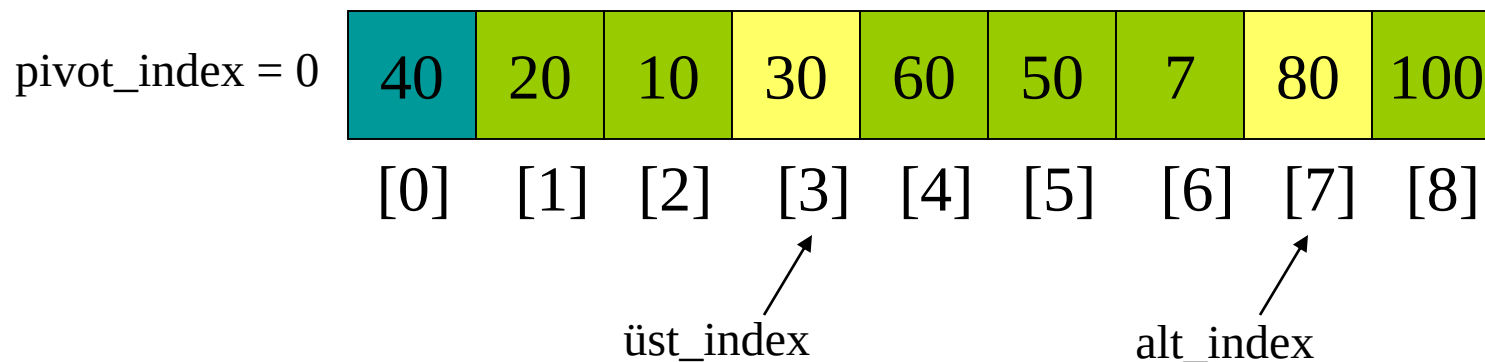
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$



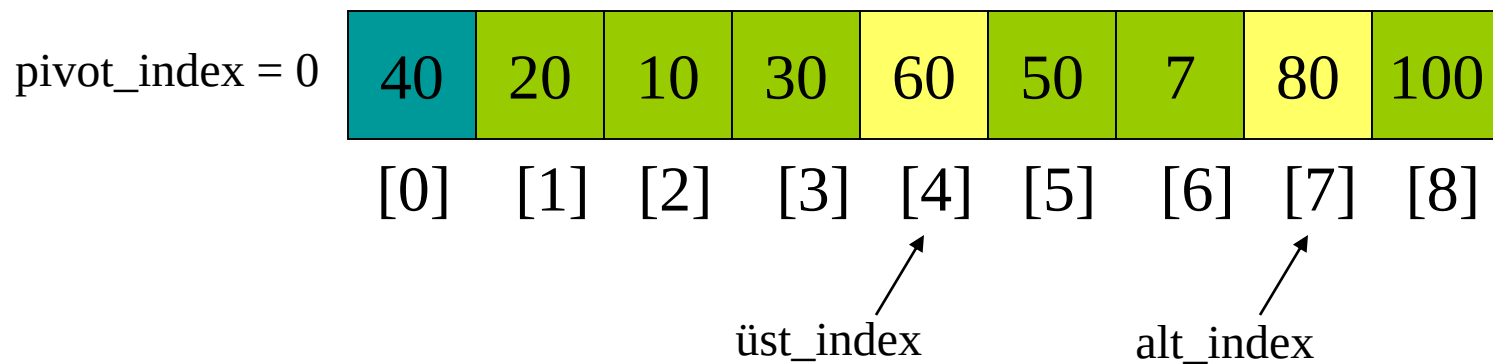
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



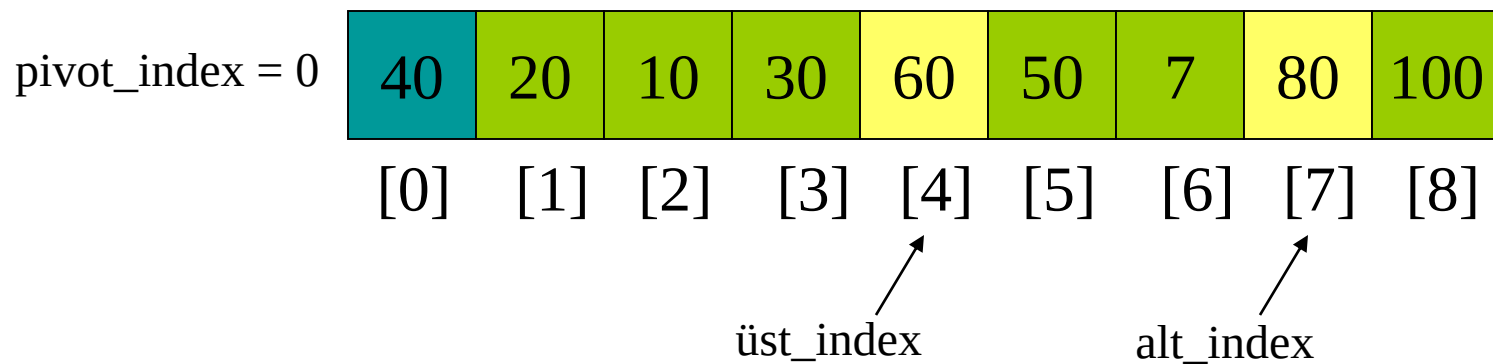
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



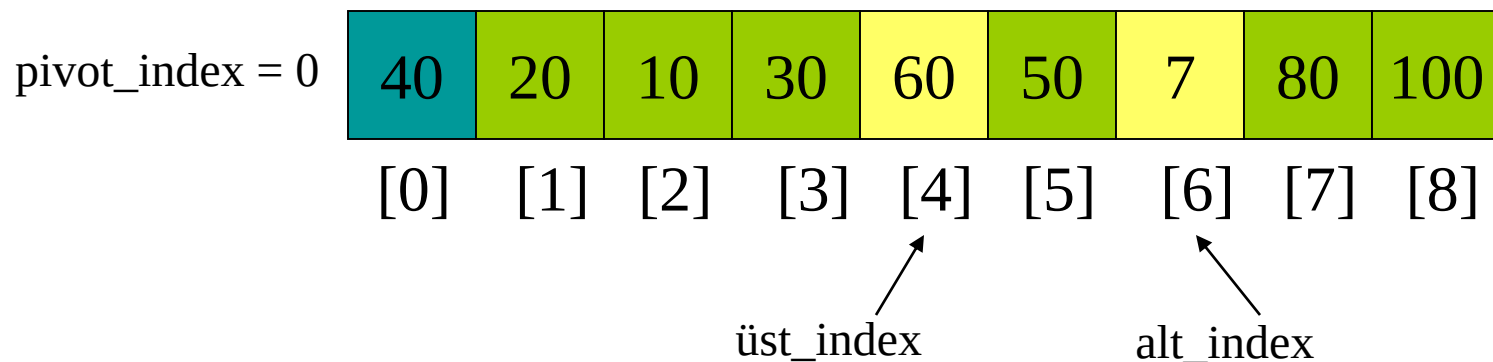
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



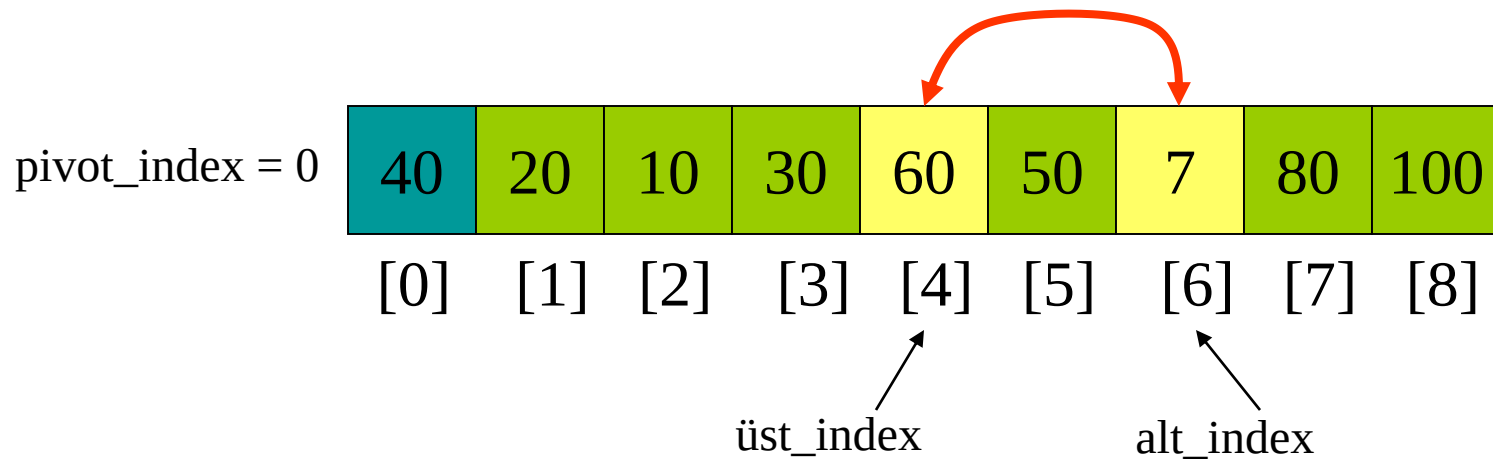
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



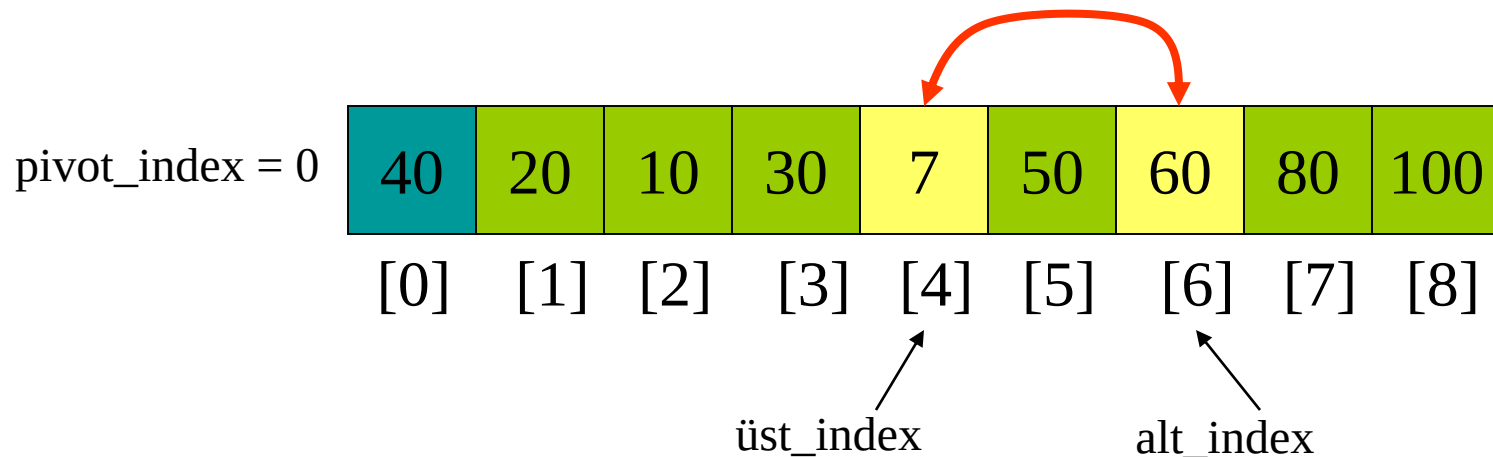
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



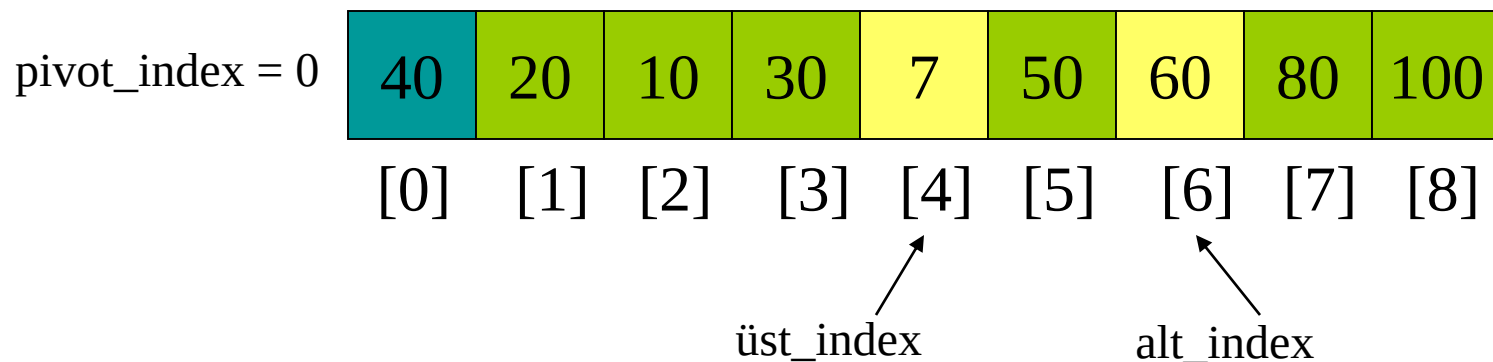
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



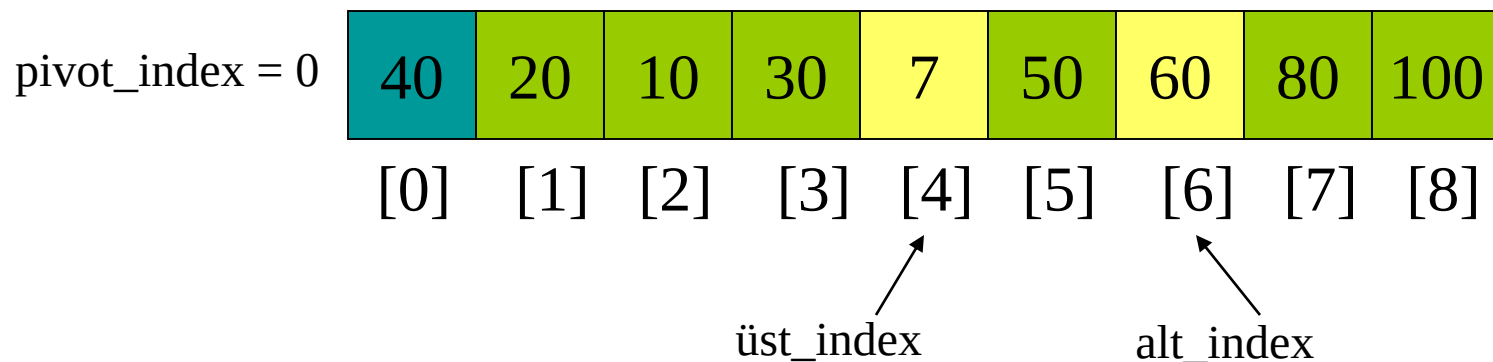
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



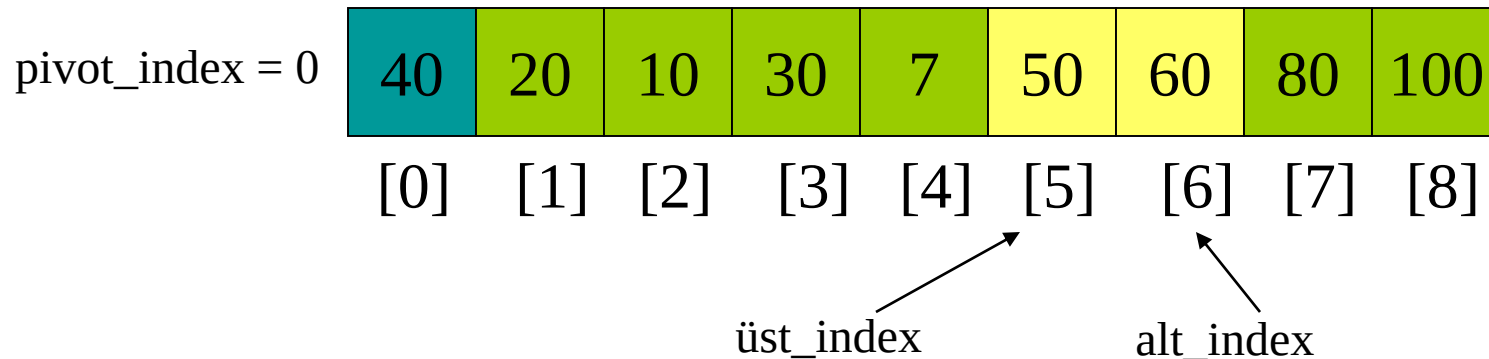
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
- 4. While $\text{alt_index} > \text{üst_index}$, go to 1.



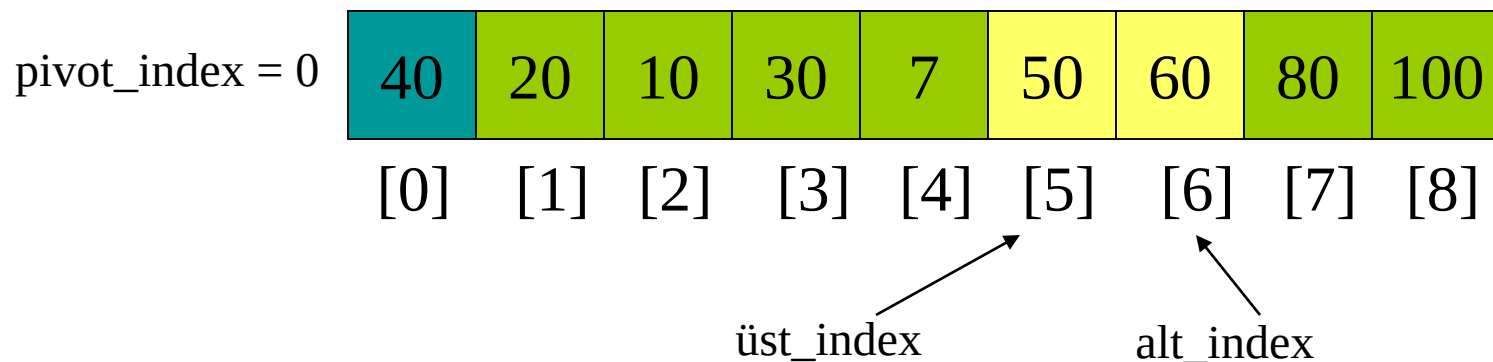
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



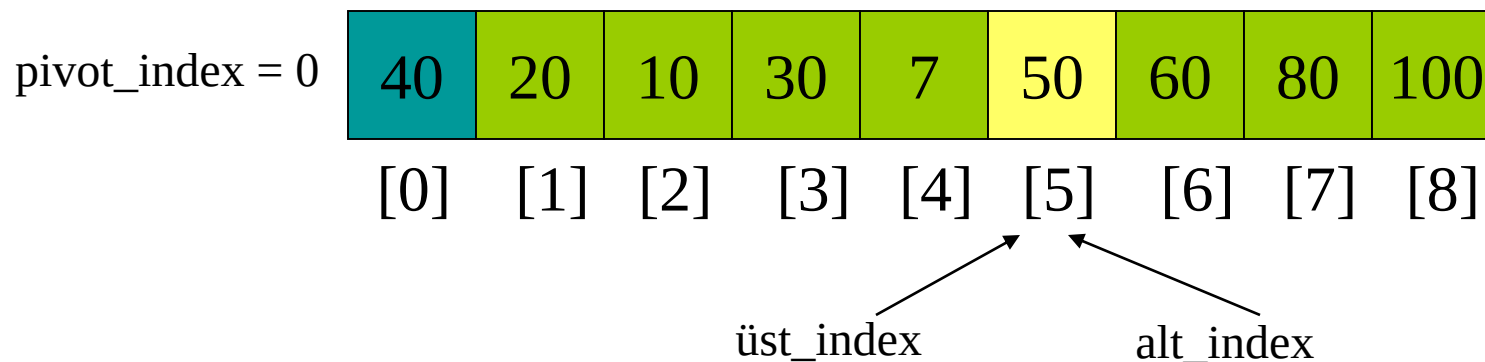
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 $++\text{üst_index}$
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 $--\text{alt_index}$
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



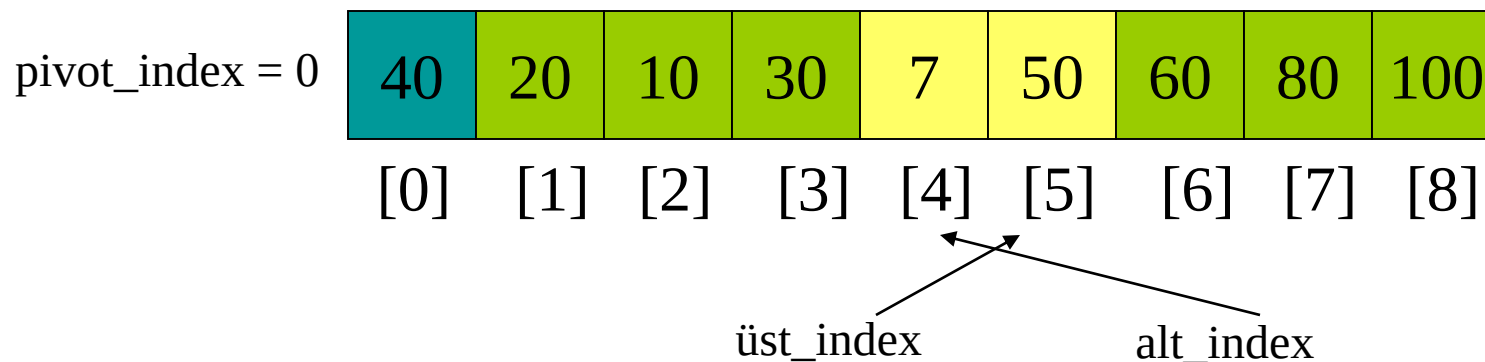
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



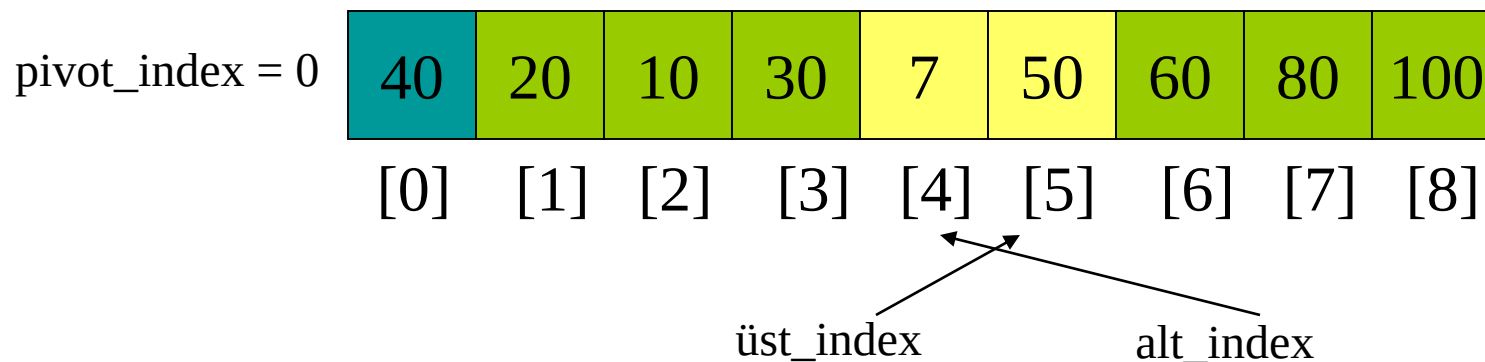
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



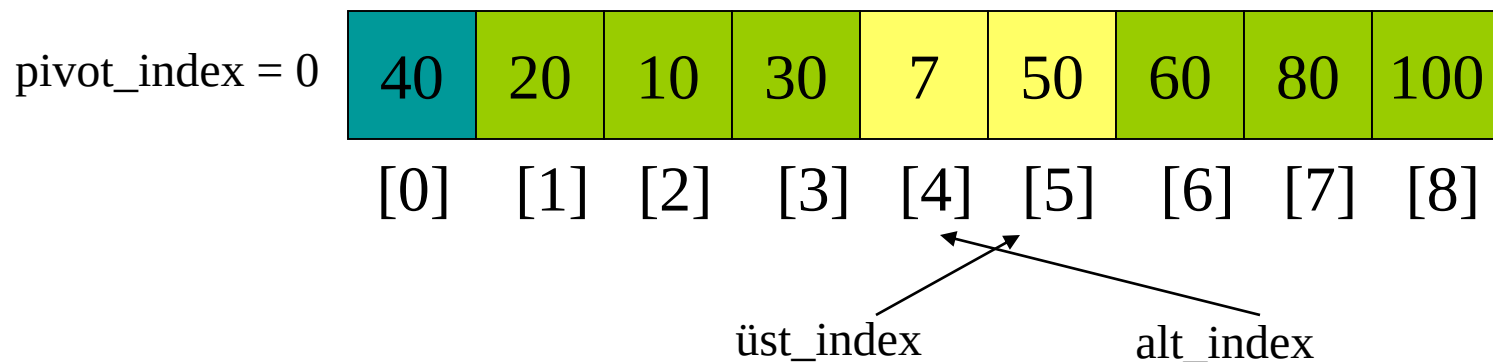
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



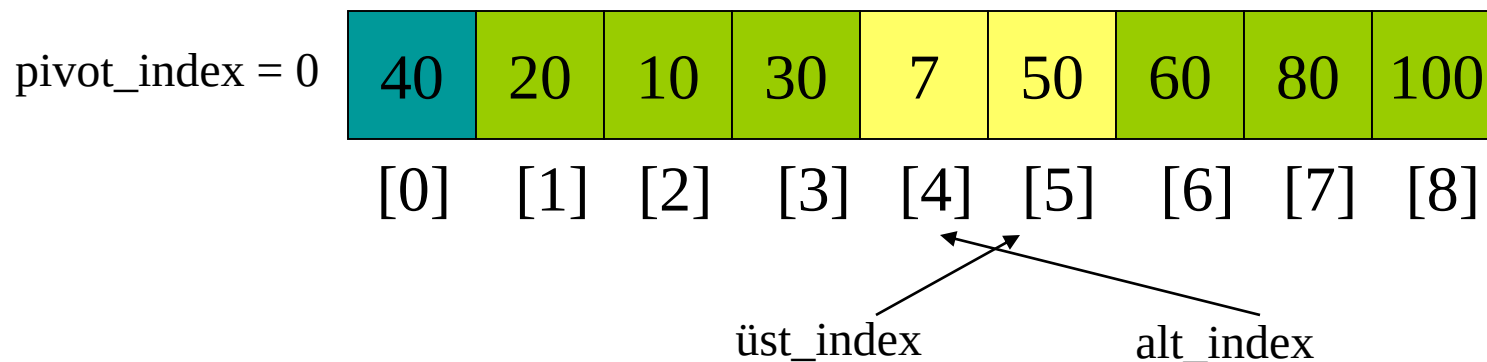
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
- 4. While $\text{alt_index} > \text{üst_index}$, go to 1.

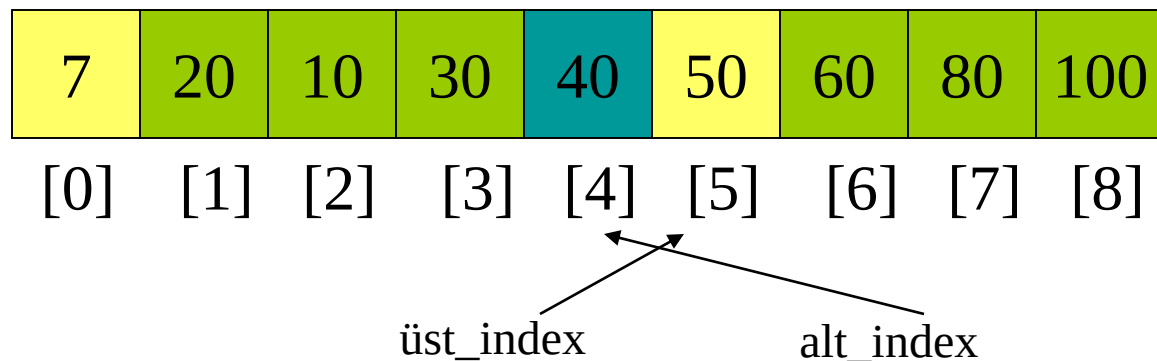


1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$

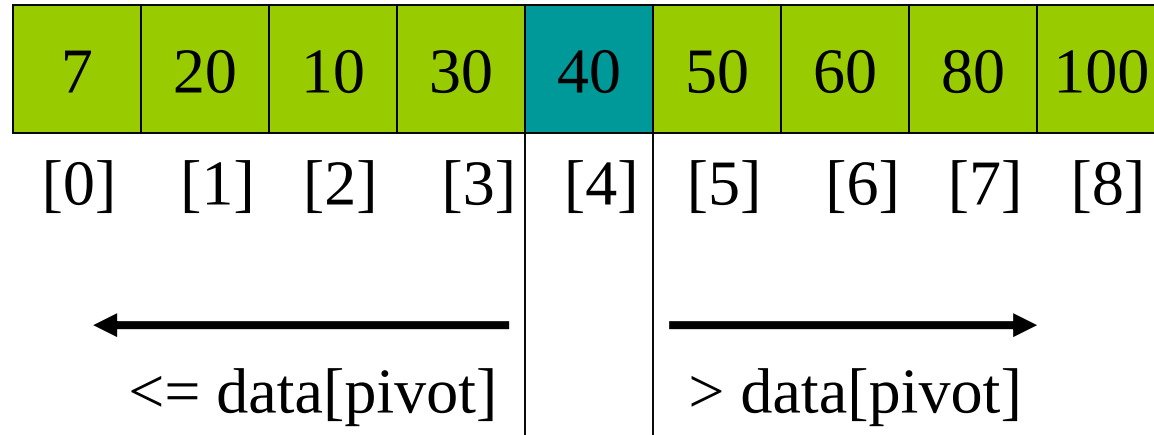


1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$

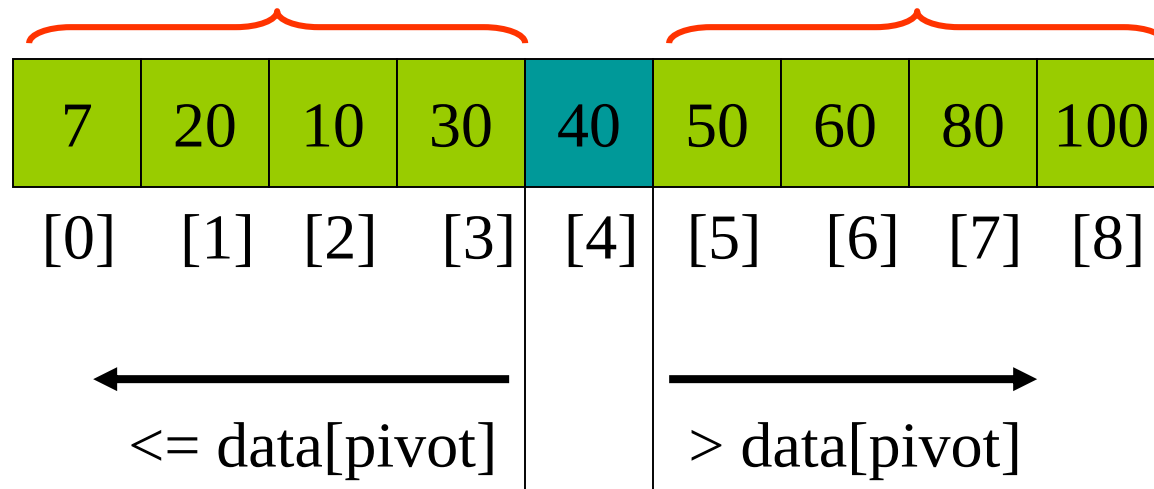
pivot_index = 4



Oluşturulan iki alt dizi

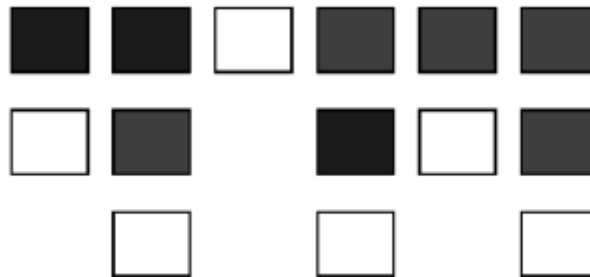


Recursion: Altdizilere Quicksort



Quicksort Algoritmasının Analizi

- Elemanlar düzensiz dağılmışsa,
- **“Best case” çalışma zamanı: $O(n \log_2 n)$**
 - Pivot eleman her zaman ortada
 - Her rekürsif çağrıda dizi iki eş parçaya ayrılmakta.



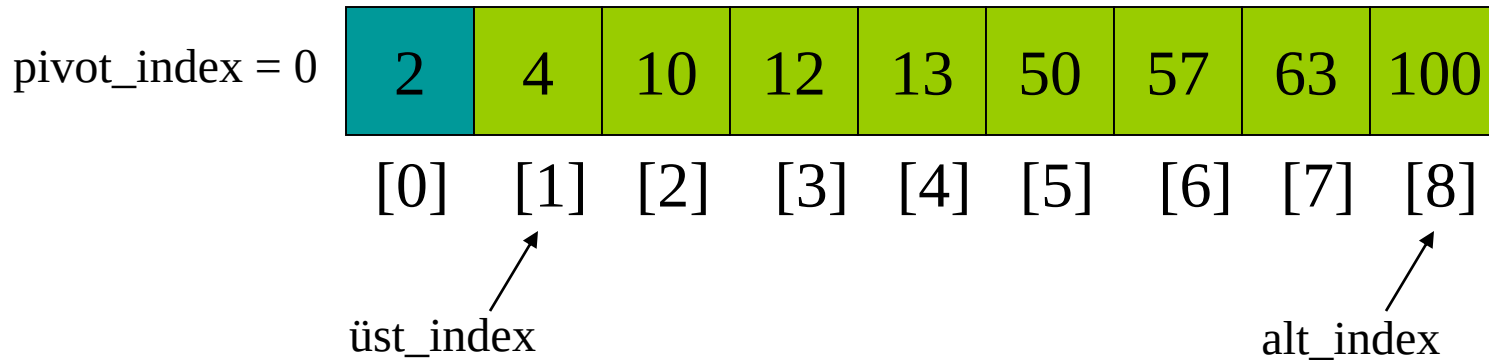
Quicksort Analysis

Worst case: $O(N^2)$

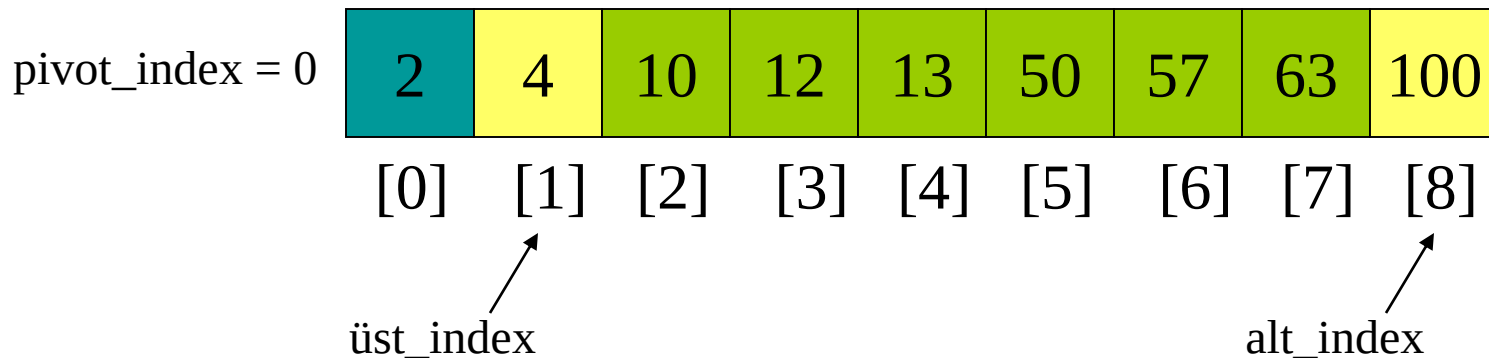
- Her rekürsif çağrıda pivot en küçük elemandır.
- Dizi zaten sıralı olması halidir.

Quicksort: Worst Case

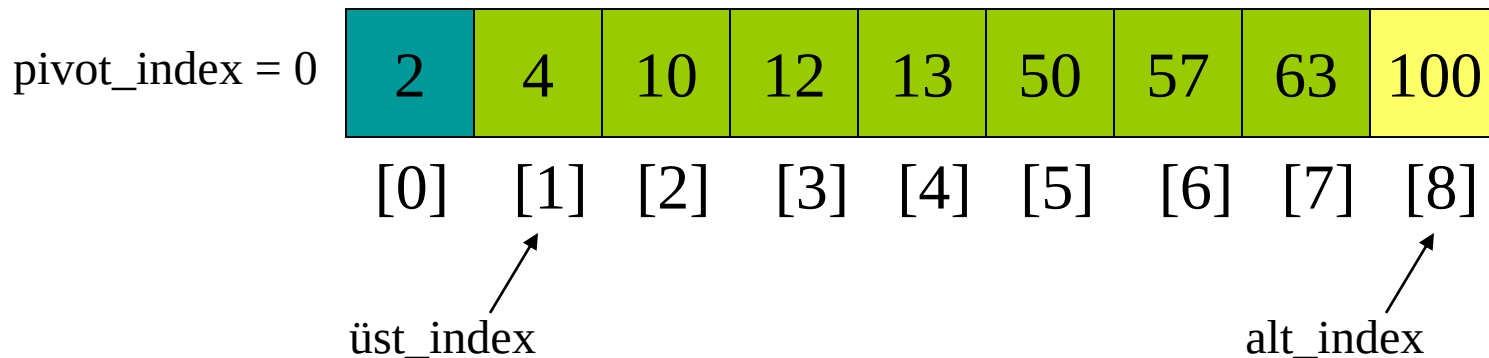
- İlk eleman pivot olarak seçilmiş olsun.
- Dizi zaten sıralı olsun



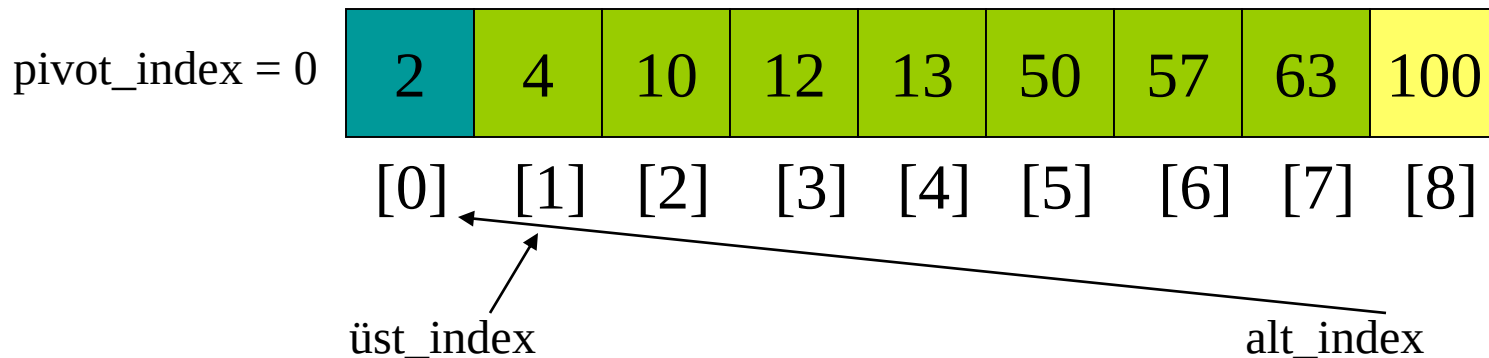
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



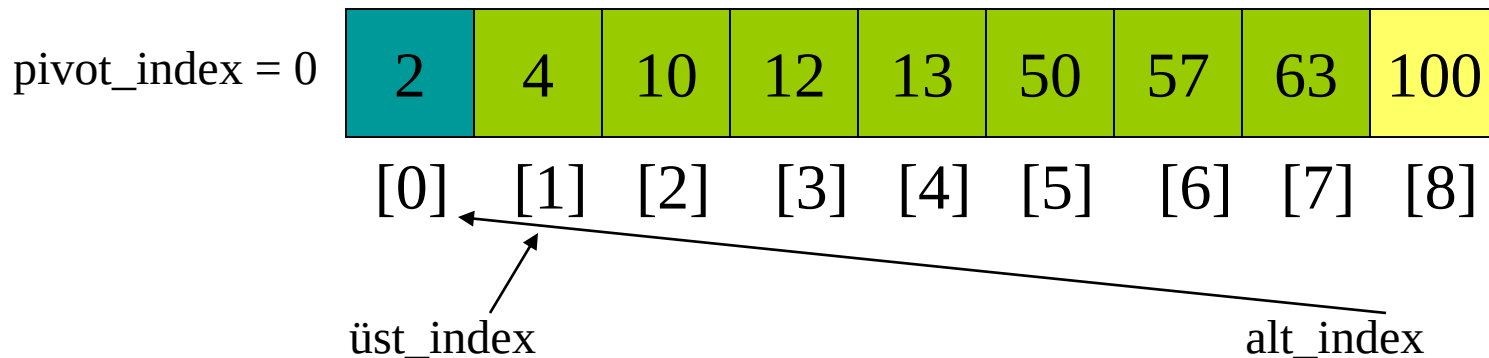
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



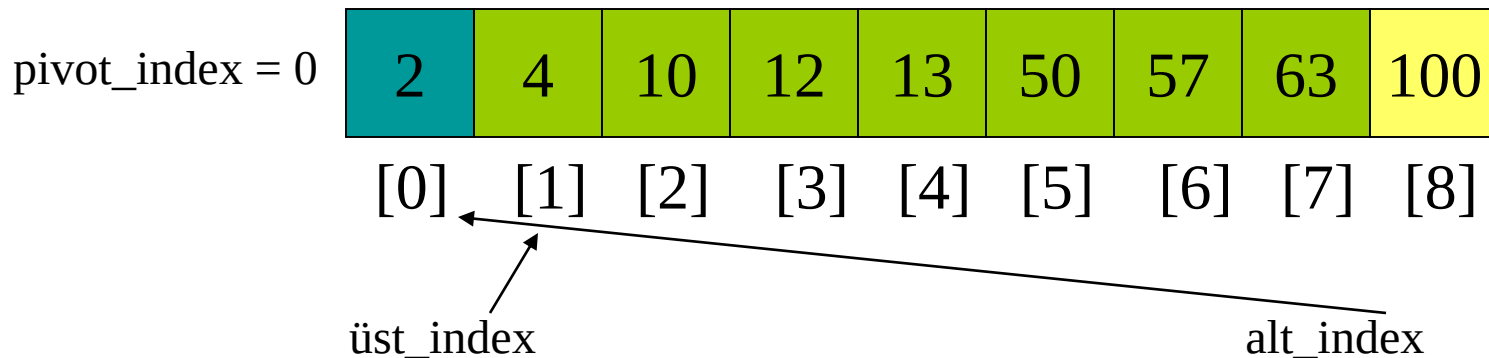
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



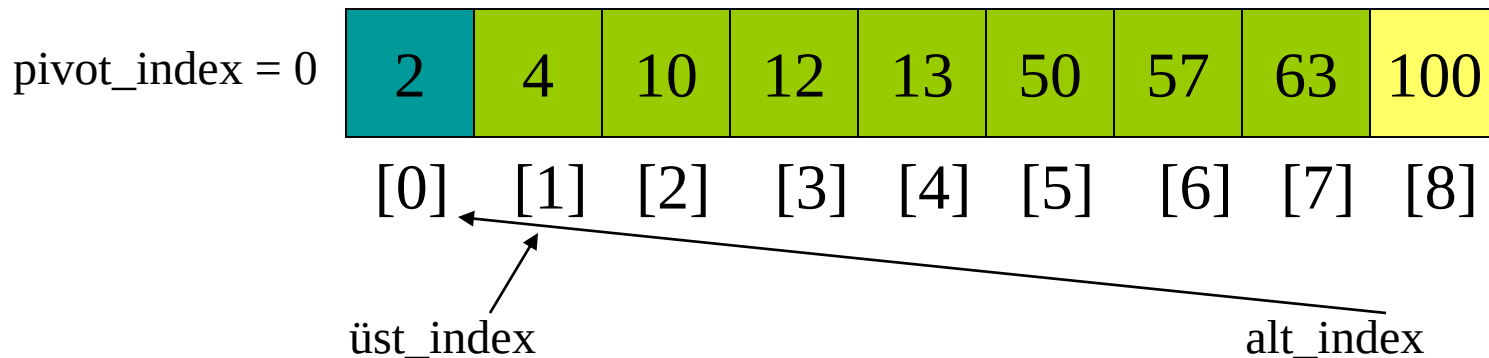
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



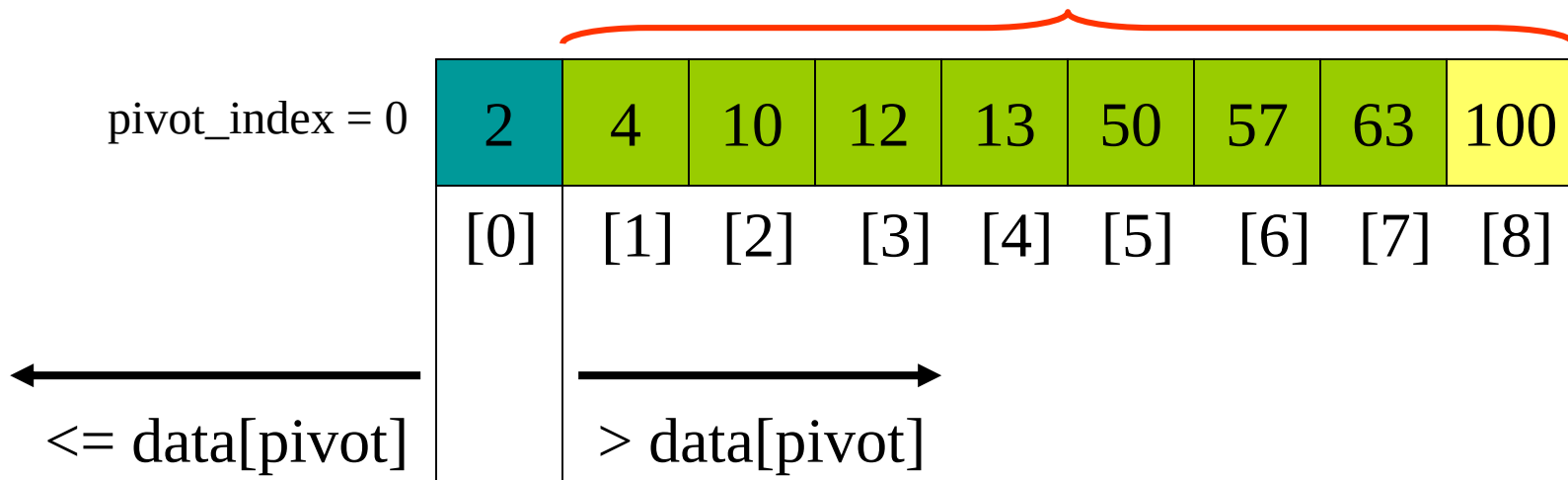
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
- 4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



$$T(n) = T(k) + T(n - k) + \alpha n$$

En kötü durumda dizinin bölünmesi 1 ve n-1 elemanlı biçiminde olacaktır.

$$T(n) = T(1) + T(n - 1) + \alpha n$$

$$T(n) = T(n - 1) + T(1) + \alpha n = [T(n - 2) + T(1) + \alpha(n - 1)] + T(1) + \alpha n$$

$$= T(n - 2) + 2T(1) + \alpha(n - 1 + n)$$

$$= [T(n - 3) + T(1) + \alpha(n - 2)] + 2T(1) + \alpha(n - 1 + n)$$

$$= T(n - 3) + 3T(1) + \alpha(n - 2 + n - 1 + n)$$

$$= [T(n - 4) + T(1) + \alpha(n - 3)] + 3T(1) + \alpha(n - 2 + n - 1 + n)$$

$$= T(n - 4) + 4T(1) + \alpha(n - 3 + n - 2 + n - 1 + n)$$

$$= T(n - i) + iT(1) + \alpha(n - i + 1 + \dots + n - 2 + n - 1 + n)$$

$$= T(n - i) + i.T(1) + \alpha(\sum_{j=0}^{i-1} (n - j))$$

$$= T(1) + (n-1)T(1) + \alpha(\sum_{j=0}^{n-2} (n - j))$$

$$\begin{matrix} \underline{n-i=1} \\ \underline{i=n-1} \end{matrix}$$

$$\sum_{j=0}^{n-2} j = \sum_{j=1}^{n-2} j = (n-2)(n-1)/2$$

Karesel
çalışma
zamanı

Eniyi Durum Analizi

En iyi durumda dizi her defasında tam olarak ikiye bölünecektir.

$$T(n) = 2T(n/2) + \alpha n = 2(2T(n/4) + \alpha n/2) + \alpha n$$

$$T(n/2) = 2T(n/4) + \alpha n/2$$

$$T(n) = 2T(n/2) + \alpha n$$

$$= 2^2T(n/4) + 2\alpha n$$

$$= 2^2(2T(n/8) + \alpha n/4) + 2\alpha n$$

$$= 2^3T(n/8) + 3\alpha n$$

$$= 2^kT(n/2^k) + k\alpha n$$

$$n = 2^k$$

$$n/2^k < 1$$

$$k = \log n$$

$$T(n) = nT(1) + \alpha n \log n, \text{ which is } O(n \log n).$$

Quicksort Algoritmasının Çalışma Zamanı Analizi

- **Best case** : $O(n \log_2 n)$
- **Worst case** : $O(n^2)$
- **Average case** : $O(n \log_2 n)$

Quicksort

QUICKSORT(A, p, r)

if $p < r$

 then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT(A, p, $q-1$)

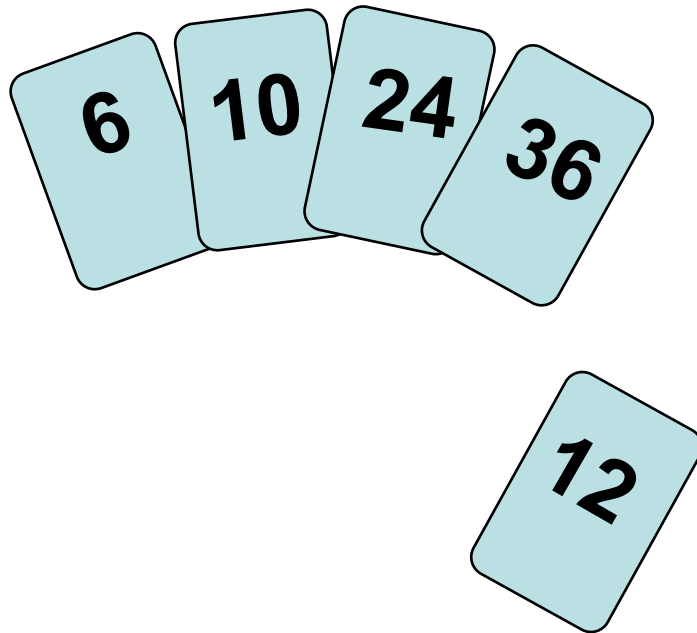
 QUICKSORT(A, $q+1$, r)

Algoritma rekürsif yapıda olduğundan bilgisayarın yığıtı yoğun bir biçimde kullanılmaktadır.

Insertion Sort (Azalt&Yönet)

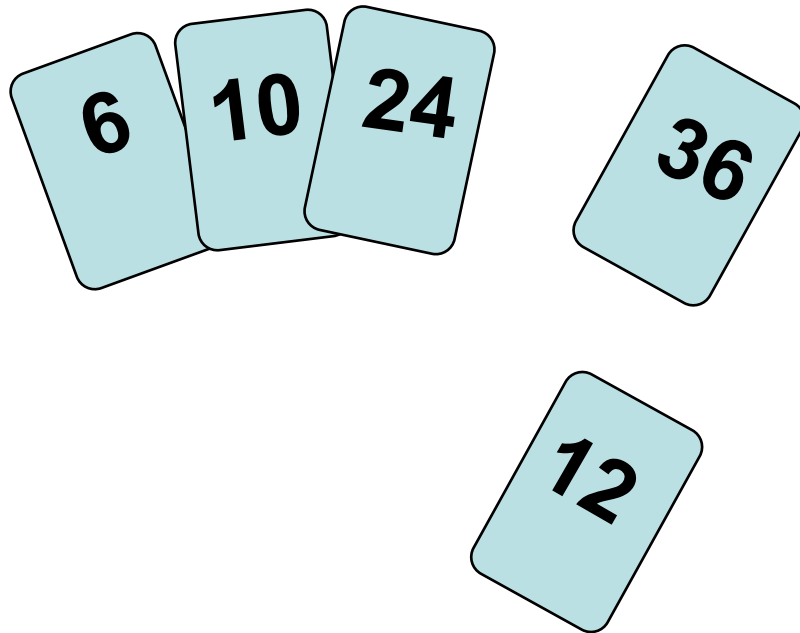
- Fikir: Oyun kartlarını sıralamaya benzer.
 - Sol elimiz boş ve masadaki kartlar resimleri aşağıda (tersyüz) olarak başlarız.
 - Masadan bir kart alırız ve onu sol elimizde uygun yere yerleştiririz.
 - Elimizdeki kartların herbiriyle sağdan sola karşılaştırırız.
 - Sol elimizde tuttuğumuz kartlar sıralıdır.
 - Bu kartlar orijinal olarak masadaki destenin en üstündeki kartlar idi.

Insertion Sort

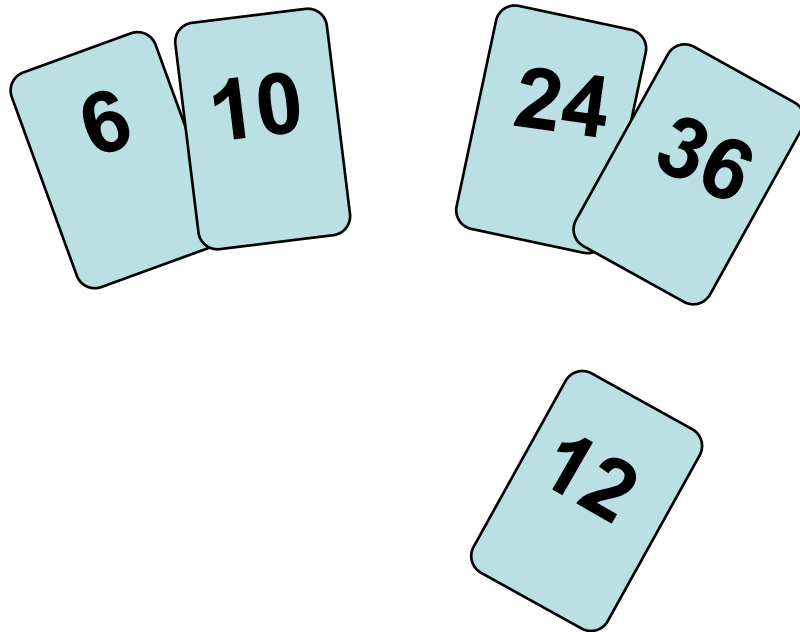


**12'yi eklemek için önce
36'yı ve daha sonra 24'ü
sağa kaydırmamız
gerekmektedir.**

Insertion Sort



Insertion Sort



Insertion Sort

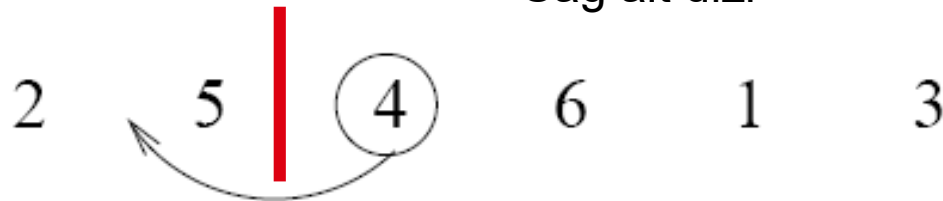
Giriş Dizisi

5 2 4 6 1 3

Her adımda,dizi iki alt diziye bölünür.

Sol alt dizi

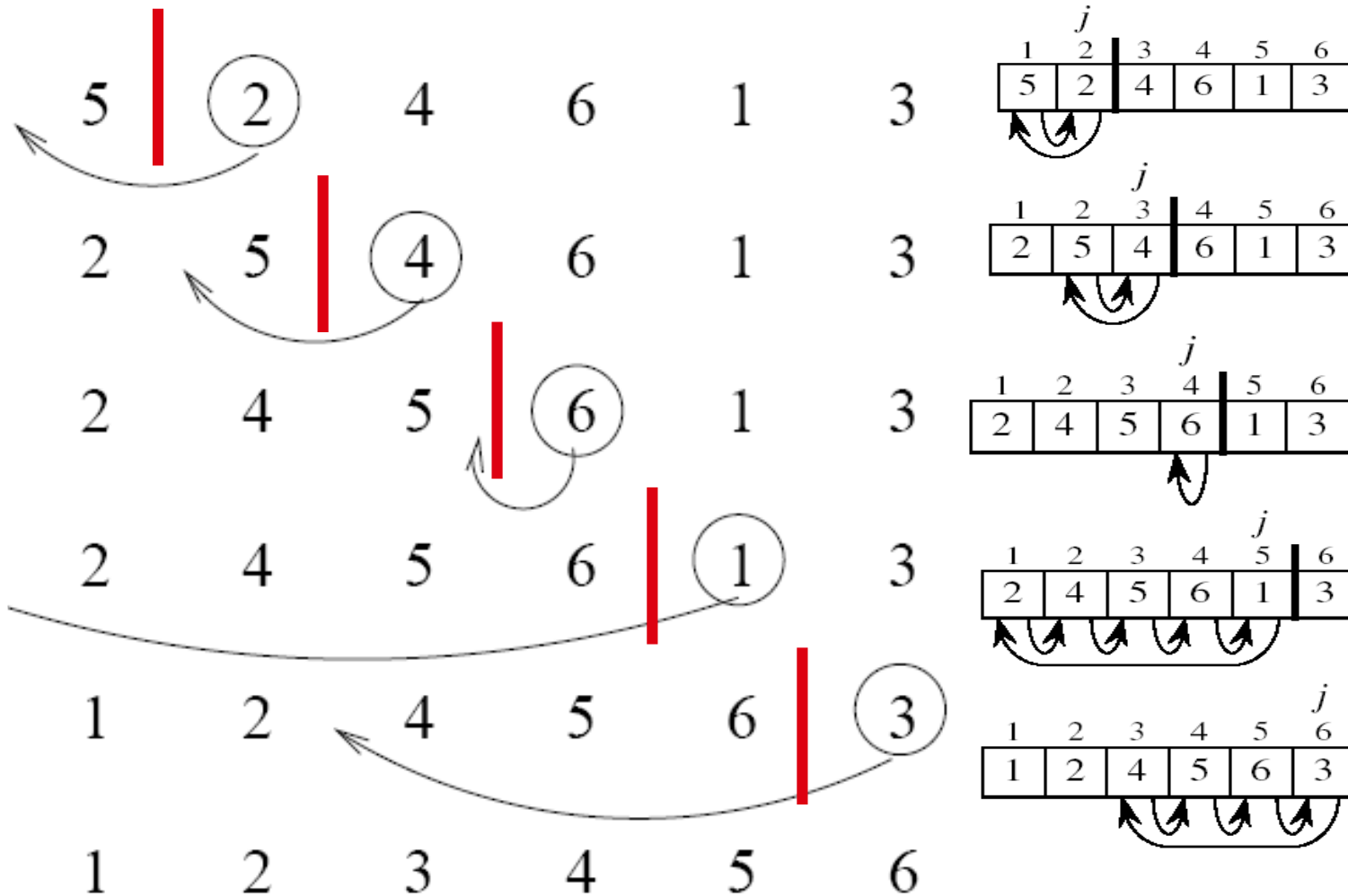
Sağ alt dizi



Sıralı

sırasız

Insertion Sort



INSERTION-SORT

Alg.: INSERTION-SORT(A)

for $j \leftarrow 2$ **to** n

do $key \leftarrow A[j]$

▷ $A[j]$ sıralı $A[1 \dots j-1]$ dizisine ekle

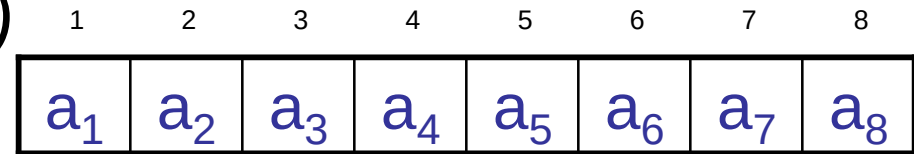
$i \leftarrow j - 1$

while $i > 0$ and $A[i] > key$

do $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow key$



- Insertion sort elemanları dizi içinde sıralar.

Insertion Sort Döngü İçerikleri

Alg.: INSERTION-SORT(A)

for $j \leftarrow 2$ **to** n

do $\text{key} \leftarrow A[j]$

$A[j]$ 'yi sıralı $A[1 \dots j-1]$ dizisine ekle

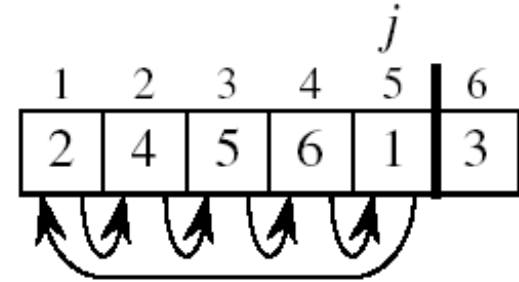
$i \leftarrow j - 1$

while $i > 0$ and $A[i] > \text{key}$

do $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow \text{key}$



Invariant: for döngüsünün başlangıcında $A[1 \dots j-1]$ içerisindeki elemanlar sıralıdır.

Döngülerin çalışması

- Döngü değişkenlerinin sınanması «induction» gibi çalışır.
- **Döngü Başlangıcı (base case):**
 - Döngünün ilk iterasyonundan önce doğru
- **Emin olma (inductive step):**
 - Döngünün bir iterasyonundan önce Doğru ise sonraki iterasyonundan önce doğru kalır.
- **Sonlanma (Termination):**
 - Döngü sonlandığında, Değişmezler algoritmanın doğru olduğunu göstermeye yardım eden yararlı özellikler verir.

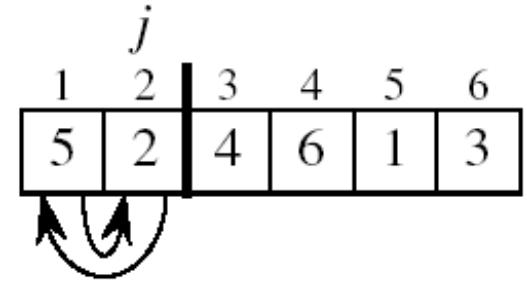
Döngü süreçleri

- Başlangıç:**

- İlk iterasyondan hemen önce, $j = 2$:

$A[1 \dots j-1] = A[1]$, alt dizisi

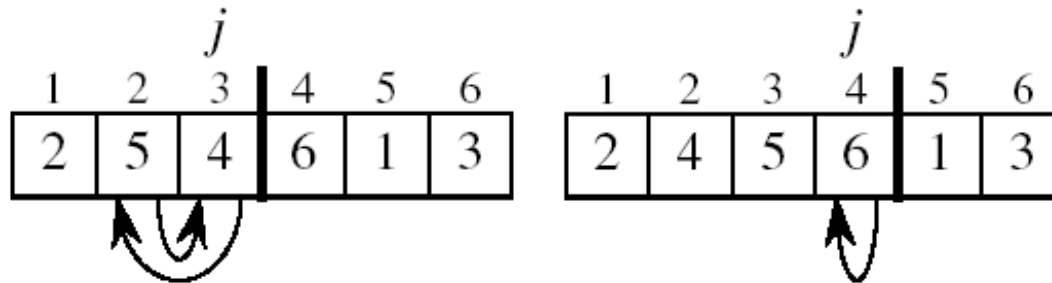
(1 elemanlı bir dizi doğal olarak sıralıdır.)



Döngü süreçleri

- **Maintenance:**

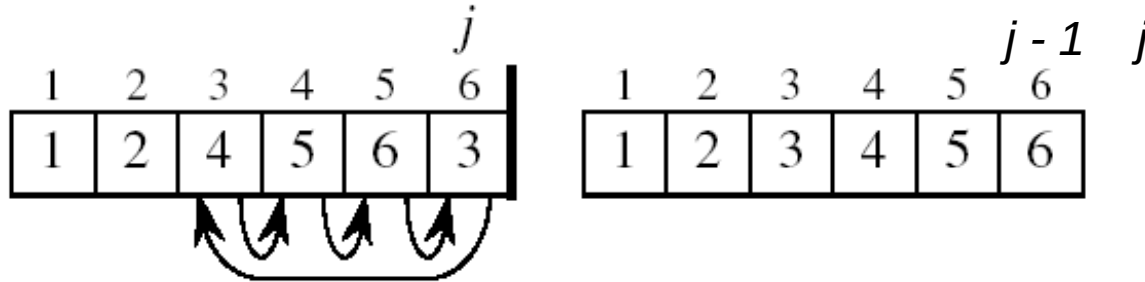
- İç **while** döngüsü , $\text{key} = A[j]$ değeri uygun pozisyona yerleşinceye kadar $A[j-1]$, $A[j-2]$, $A[j-3]$,... biçiminde hareket eder
- Bu noktada, key değeri uygun pozisyona yerleşir.



Döngü süreçleri

- **Sonlanma:**

- Dıştaki **for** döngüsü $j = n + 1 \Rightarrow j-1 = n$ olduğunda biter.
- n , loop invariant içindeki $j-1$ ile yer değiştirir.
 - $A[1 \dots n]$ alt dizisi orijinal $A[1 \dots n]$ dizisini (sıralı) içerir.



Insertion Sort

INSERTION-SORT(A)

for j ← 2 to n

do key ← A[j]

▷A[j]'yi sıralı A[1 . . j -1] dizisine ekle

i ← j - 1

while i > 0 and A[i] > key

do A[i + 1] ← A[i]

i ← i - 1

A[i + 1] ← key

cost times

c_1

n

c_2

n-1

0

n-1

c_4

n-1

c_5

$\sum_{j=2}^n t_j$

c_6

$\sum_{j=2}^n (t_j - 1)$

c_7

$\sum_{j=2}^n (t_j - 1)$

c_8

n-1

t_j : j. İterasyonda çalıştırılan while ifadesi sayısı

$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8 (n-1)$$

En İyi Durum Analizi (Best Case Analysis)

- Dizi zaten sıralıdır. “**while** $i > 0$ and $A[i] > \text{key}$ ”
 - **while** döngüsü testi ilk çalıştığında $A[i] \leq \text{key}$ olur ($i=j-1$)
 - $t_j = 1$
- $$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$$
$$= (c_1 + c_2 + c_4 + c_5 + c_8)n + (c_2 + c_4 + c_5 + c_8)$$
$$= an + b = \Theta(n)$$

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

En Kötü Durum Analizi (Worst Case Analysis)

- Dizi tersten sıralıdır. “while i > 0 and A[i] > key”
 - while döngüsü testinde her zaman $A[i] > key$ olacaktır.
 - Key j’nci pozisyonun solundaki bütün elemanlarla karşılaştırılır
(j. elmanı $\Rightarrow$ karşılaştır j-1 eleman ile $\Rightarrow t_j = j$)

$$\sum_{j=1}^n j = \frac{n(n+1)}{2} \Rightarrow \sum_{j=2}^n j = \frac{n(n+1)}{2} - 1 \Rightarrow \sum_{j=2}^n (j-1) = \frac{n(n-1)}{2} \quad \text{we have:}$$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + c_6 \frac{n(n-1)}{2} + c_7 \frac{n(n-1)}{2} + c_8(n-1)$$

$$= an^2 + bn + c$$

Karesel bir fonksiyon

- $T(n) = \Theta(n^2)$

Büyüme derecesi: n^2

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

Insertion Sort algoritmasındaki Karşılaştırma ve Yer değiştirmeler

INSERTION-SORT(A)

for $j \leftarrow 2$ to n

cost times

c_1 n

do $\text{key} \leftarrow A[j]$

c_2 $n-1$

Insert $A[j]$ 'yi sıralı diziye $A[1 \dots j-1]$

0 $n-1$

$i \leftarrow j - 1$

$\approx n^2/2$ karşılaştırma c_4 $n-1$

while $i > 0$ and $A[i] > \text{key}$

c_5 $\sum_{j=2}^n t_j$

do $A[i + 1] \leftarrow A[i]$

c_6 $\sum_{j=2}^n (t_j - 1)$

$i \leftarrow i - 1$

$\approx n^2/2$ yerdeğiştirme c_7

$\sum_{j=2}^n (t_j - 1)$

$A[i + 1] \leftarrow \text{key}$

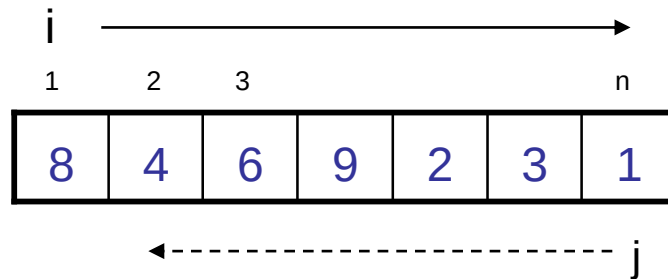
c_8 $n-1$

Genel değerlendirme

- Avantaj:
 - Zaten sıralı bir dizi için iyi bir çalışma zamanına sahiptir: $\Theta(n)$
- Dezavantaj:
 - Ortalama ve en kötü durumlarda çalışma zamanı $\Theta(n^2)$ olur.
 - $\approx n^2/2$ karşılaştırma ve yer değiştirme

Bubble Sort

- Algoritma fikri:
 - Dizinin elemanları üzerinden tekrar tekrar geçilir.
 - Sırasız komşu elemanlar yer değiştirilir.



- İşletimi kolaydır, Fakat Insertion sort algoritmasından daha yavaştır.

Example

8	4	6	9	2	3	1
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	6	9	2	1	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	6	9	1	2	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	6	1	9	2	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	4	1	6	9	2	3
---	---	---	---	---	---	---

$i = 1$ ←----- j

8	1	4	6	9	2	3
---	---	---	---	---	---	---

$i = 1$ j

1	8	4	6	9	2	3
---	---	---	---	---	---	---

$i = 1$ j

1	8	4	6	9	2	3
---	---	---	---	---	---	---

$i = 2$ j

1	2	8	4	6	9	3
---	---	---	---	---	---	---

$i = 3$ j

1	2	3	8	4	6	9
---	---	---	---	---	---	---

$i = 4$ j

1	2	3	4	8	6	9
---	---	---	---	---	---	---

$i = 5$ j

1	2	3	4	6	8	9
---	---	---	---	---	---	---

$i = 6$ j

1	2	3	4	6	8	9
---	---	---	---	---	---	---

$i = 7$

j

Bubble Sort

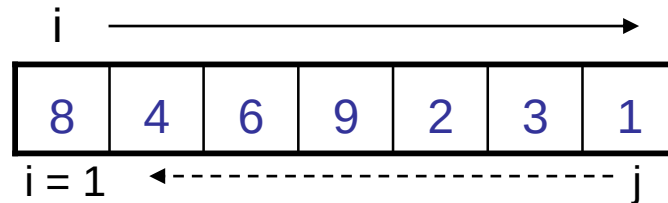
Alg.: BUBBLESORT(A)

for $i \leftarrow 1$ **to** $\text{length}[A]$

do for $j \leftarrow \text{length}[A]$ **downto** $i + 1$

do if $A[j] < A[j - 1]$

then exchange $A[j] \leftrightarrow A[j - 1]$



Bubble-Sort Çalışma Zamanı

Alg.: BUBBLESORT(A)

for $i \leftarrow 1$ to $\text{length}[A]$ c_1

do for $j \leftarrow \text{length}[A]$ downto $i + 1$ c_2

Karşılaştırma: $\approx n^2/2$

do if $A[j] < A[j - 1]$ c_3

Yerdeğiştirme: $\approx n^2/2$

then exchange $A[j] \leftrightarrow A[j - 1]$ c_4

$$T(n) = c_1(n+1) + c_2 \sum_{i=1}^n (n-i+1) + c_3 \sum_{i=1}^n (n-i) + c_4 \sum_{i=1}^n (n-i)$$

$$= \Theta(n) + (c_2 + c_3 + c_4) \sum_{i=1}^n (n-i)$$

$$\text{where } \sum_{i=1}^n (n-i) = \sum_{i=1}^n n - \sum_{i=1}^n i = n^2 - \frac{n(n+1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

$$\text{Thus, } T(n) = \Theta(n^2)$$

Selection Sort

- **Algoritmanın çalışması:**
 - Dizinin en küçük elemanını bul
 - Bu en küçük elemanı birinci pozisyonndaki elemanla yer değiştir.
 - İkinci en küçük elemanı bul ve bunu ikinci pozisyonndaki elemanla yer değiştir.
 - Dizi sıralanıncaya kadar bu işlemlere devam et.
- **Dezavantaj:**
 - Çalışma zamanı dizinin düzenine çok az bağlı. Eleman sayısı arttıkça her durumda çalışma zamanı hızlı bir artış gösterecektir.

Example

8	4	6	9	2	3	1
---	---	---	---	---	---	---

1	4	6	9	2	3	8
---	---	---	---	---	---	---

1	2	6	9	4	3	8
---	---	---	---	---	---	---

1	2	3	9	4	6	8
---	---	---	---	---	---	---

1	2	3	4	9	6	8
---	---	---	---	---	---	---

1	2	3	4	6	9	8
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

Selection Sort

Alg.: SELECTION-SORT(A)

$n \leftarrow \text{length}[A]$

for $j \leftarrow 1$ **to** $n - 1$

do $ek \leftarrow j$

for $i \leftarrow j + 1$ **to** n

do if $A[i] < A[ek]$

then $ek \leftarrow i$

exchange $A[j] \leftrightarrow A[ek]$

8	4	6	9	2	3	1
---	---	---	---	---	---	---

Selection Sort Algoritmasının Analizi

Alg.: SELECTION-SORT(A)

cost times

$n \leftarrow \text{length}[A]$

c_1 1

for $j \leftarrow 1$ **to** $n - 1$

c_2 n

do $ek \leftarrow j$

c_3 $n-1$

$\approx n^2/2$

karşılaştırma

for $i \leftarrow j + 1$ **to** n

c_4 $\sum_{j=1}^{n-1} (n - j + 1)$

do if $A[i] < A[ek]$

c_5 $\sum_{j=1}^{n-1} (n - j)$

$\approx n$

yerdeğiştirme

then $ek \leftarrow i$

c_6 $\sum_{j=1}^{n-1} (n - j)$

exchange $A[j] \leftrightarrow A[ek]$

c_7 $n-1$

$$T(n) = c_1 + c_2 n + c_3 (n-1) + c_4 \sum_{j=1}^{n-1} (n - j + 1) + c_5 \sum_{j=1}^{n-1} (n - j) + c_6 \sum_{j=2}^{n-1} (n - j) + c_7 (n-1) = \Theta(n^2) \quad 25$$

Yinelemeli (Iterative) Algoritmalar

Analiz

ADD-1(*n*: nonnegative integer)

```
1  a ← 0
2  for i ← 1 to n
3      a ← a + i
4  return a
```

$$\sum_{i=1}^n 1 = n \quad \Theta(n)$$

ADD-3(*n*: nonnegative integer)

```
1  a ← 0
2  for i ← 1 to n
3      for j ← 1 to n
4          a ← a + 1
5  return a
```

$$\sum_{i=1}^n \sum_{j=1}^n 1 = \sum_{i=1}^n n = n^2 \quad \Theta(n^2)$$

ADD-GENERALISED(n : nonnegative integer)

1 for $i_1 \leftarrow 1$ to n

2 for $i_2 \leftarrow 1$ to n

3 ...

4 for $i_k \leftarrow 1$ to n

5 expression $\Theta(1)$

$\Theta(n^k)$

ADD-4(n : nonnegative integer)

```
1  a ← 0
2  for i ← 1 to n
3      for j ← i to n
4          a ← a + 1
5  return a
```

$$\sum_{i=1}^n \sum_{j=i}^n 1 = \sum_{i=1}^n \left(\underbrace{\sum_{j=1}^n 1}_n - \underbrace{\sum_{j=1}^{i-1} 1}_{i-1} \right) = \sum_{i=1}^n (n - i + 1) =$$

$$\sum_{i=1}^n (n + 1) - \sum_{i=1}^n i = n(n + 1) - \frac{n(n + 1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n = \Theta(n^2)$$

ADD-5(*n*: nonnegative integer)

```
1  a ← 0
2  for i ← 1 to n
3      for j ← i + 1 to n
4          a ← a + 1
5  return a
```

$$\sum_{i=1}^n \sum_{j=i+1}^n 1 = \sum_{i=1}^n \left(\underbrace{\sum_{j=1}^n 1}_n - \underbrace{\sum_{j=1}^i 1}_i \right) = \sum_{i=1}^n (n - i) = \sum_{i=1}^n (n) - \sum_{i=1}^n i =$$

$$n^2 - \frac{n(n+1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n = \Theta(n^2)$$

A2(n: positive integer)

```

1  a ← 0
2  for i ← 1 to n-1
3      for j ← i+1 to n
4          for k ← 1 to j
5              a ← a+1
6  return a

```

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n \sum_{k=1}^j 1 = \sum_{i=1}^{n-1} \sum_{j=i+1}^n j = \sum_{i=1}^{n-1} \left(\sum_{j=1}^n j - \sum_{j=1}^i j \right)$$

$$= \sum_{i=1}^{n-1} \left(\frac{1}{2}n(n+1) - \frac{1}{2}i(i+1) \right) = \sum_{i=1}^{n-1} \left(\frac{1}{2}n(n+1) \right) - \frac{1}{2} \sum_{i=1}^{n-1} (i^2 + i)$$

$$= \frac{1}{2}n(n+1)(n-1) - \frac{1}{2} \sum_{i=1}^{n-1} i^2 - \frac{1}{2} \sum_{i=1}^{n-1} i$$

$$= \frac{1}{2}n(n-1)(n+1) - \frac{1}{12}n(n-1)(2n-1) - \frac{1}{4}n(n-1)$$

$$= \frac{1}{2}n(n-1) \left(n+1 - \frac{1}{6}(2n-1) - \frac{1}{2} \right)$$

$$= \frac{1}{12}n(n-1)(6n+3-2n+1) = \frac{1}{12}n(n-1)(4n+4) = \frac{1}{3}n(n-1)(n+1)$$

$\Theta(n^3)$

$$\sum_{i=1}^n i^2 = \frac{1}{6}n(n+1)(2n+1)$$

$$\sum_{i=1}^{n-1} i^2 = \frac{1}{6}(n-1)n(2n-1)$$

$$\sum_{i=1}^{n-1} i = \frac{1}{2}n(n-1)$$

Verilen Algoritmanın çalışma zamanı büyüme hızı nedir?

P 1($a_1, a_2, \dots, a_n$: array of positive integers)

1 $s \leftarrow 0$

2 **for** $i \leftarrow 1$ **to** $n - 4$

3 **for** $j \leftarrow i$ **to** $i + 4$

4 **for** $k \leftarrow i$ **to** j

5 $s \leftarrow s + a_i$

En dıştaki döngü $n-4$ defa çalışır.

Ortadaki döngü tam olarak 5 defa çalışır.

En içteki döngü j 'nin $i, i + 1, i + 2, i + 3$ ve $i + 4$ 'e eşit değerleri için sırasıyla 1, 2, 3, 4 ve 5 kez ve toplam 15 kez yürütülür

$\Theta(n)$

P 1($a_1, a_2, \dots, a_n$: array of positive integers)

1 $s \leftarrow 0$

2 **for** $i \leftarrow 1$ **to** $n - 4$

3 **for** $j \leftarrow i$ **to** $i + 4$

4 **for** $k \leftarrow i$ **to** j

5 $s \leftarrow s + a_i$

$$\sum_{i=1}^{n-4} \sum_{j=1}^{i+4} \sum_{k=i}^j 1$$

$$\sum_{k=i}^j 1 = \begin{cases} j - i + 1, & \text{if } j \geq i \\ 0, & \text{else} \end{cases}$$

$$\sum_{i=1}^{n-4} \left(\underbrace{\sum_{j=1}^{i-1} \sum_{k=i}^j 1}_0 + \sum_{j=i}^{i+4} \sum_{k=i}^j 1 \right) = \sum_{i=1}^{n-4} \sum_{j=i}^{i+4} (j - i + 1) =$$

$$= \sum_{i=1}^{n-4} ((i - i + 1) + (i + 1 - i + 1) + (i + 2 - i + 1) + (i + 3 - i + 1) + (i + 4 - i + 1))$$

$$= \sum_{i=1}^{n-4} (1 + 2 + 3 + 4 + 5) = 15(n - 4)$$